Experiment No: 7			
Course Name:	Web Programming Laboratory (216U01L405)	**Semester:**	IV
Date of Performance:	**07/ 04/ 2026**	**DIV/ Batch No:**	**B2**
Student Name:	**Ashwera Hasan**	**Roll No:**	**16010124107**

Title: Create a web application that implements session management and cookies for user authentication and state maintenance.

Aim of the Experiment: The primary aim of this project is to develop a secure and efficient web application that implements robust session management and cookie handling for user authentication and state maintenance. This includes ensuring user data integrity, providing a seamless user experience, and implementing security features to mitigate common vulnerabilities.

Objectives for the Experiment:

- Develop a secure user registration system that allows users to create accounts by collecting and storing their **credentials** (username, email, and password) in a MySQL database, ensuring passwords are **hashed** for security.
- Create a robust user login mechanism that verifies user credentials against the stored data, initiating a session for authenticated users to maintain their state across different pages of the application.
- Implement session management using PHP sessions to track user login status and facilitate navigation within the application while maintaining user state during their session.
- Set up cookies to enable "remember me" functionality, allowing users to stay logged in across sessions for a defined period, enhancing user experience and convenience.
- Incorporate validation techniques for user input during both registration and login processes to prevent SQL injection and cross-site scripting (XSS) attacks, ensuring a secure application environment (if feasible).

COs to be achieved:

1. Design dynamic web pages using various HTML tags.
2. Use CSS to prepare the layout of web pages.
3. Apply JavaScript for validation in client-side programming.
4. Integrate server-side pages using PHP.
5. Apply database operations by integrating SQL queries and session variables.

Books/ Journals/ Websites references:

1. Students should write

Software / Tools Required

- XAMPP / WAMP / MAMP
- PHP
- MySQL
- phpMyAdmin
- Code Editor (VS Code / Sublime / Notepad++/ any other)
- Web Browser

Methodology:

1. Start XAMPP server and MySQL service.
2. Create database and users table in phpMyAdmin.
3. Design registration and login forms using HTML.
4. Store user data securely with password hashing.
5. Implement login verification using PHP.
6. Use sessions to maintain user login state.
7. Implement cookies for "remember me" functionality.
8. Validate user inputs and handle errors.
9. Test application and display results.

Theory:

Session management allows tracking user activity across multiple pages using server-side storage, while cookies store small data on the client side. PHP sessions help maintain authentication state, and cookies enable features like "remember me." Secure implementation includes password hashing, input validation, and protection against session hijacking and XSS attacks.

Welling, L., & Thomson, L. (2017). *PHP and MySQL web development* (5th ed.). Addison-Wesley.

Nixon, R. (2018). *Learning PHP, MySQL & JavaScript* (5th ed.). O'Reilly Media.

PHP Documentation. (2024). *Sessions and cookies in PHP for web applications*. https://www.php.net

MySQL Documentation. (2024). *Database management for web-based inventory systems*. https://dev.mysql.com

Problem statement:

Design a user authentication system for an inventory management application where users can register, log in, and maintain session state using PHP sessions and cookies with secure validation mechanisms.

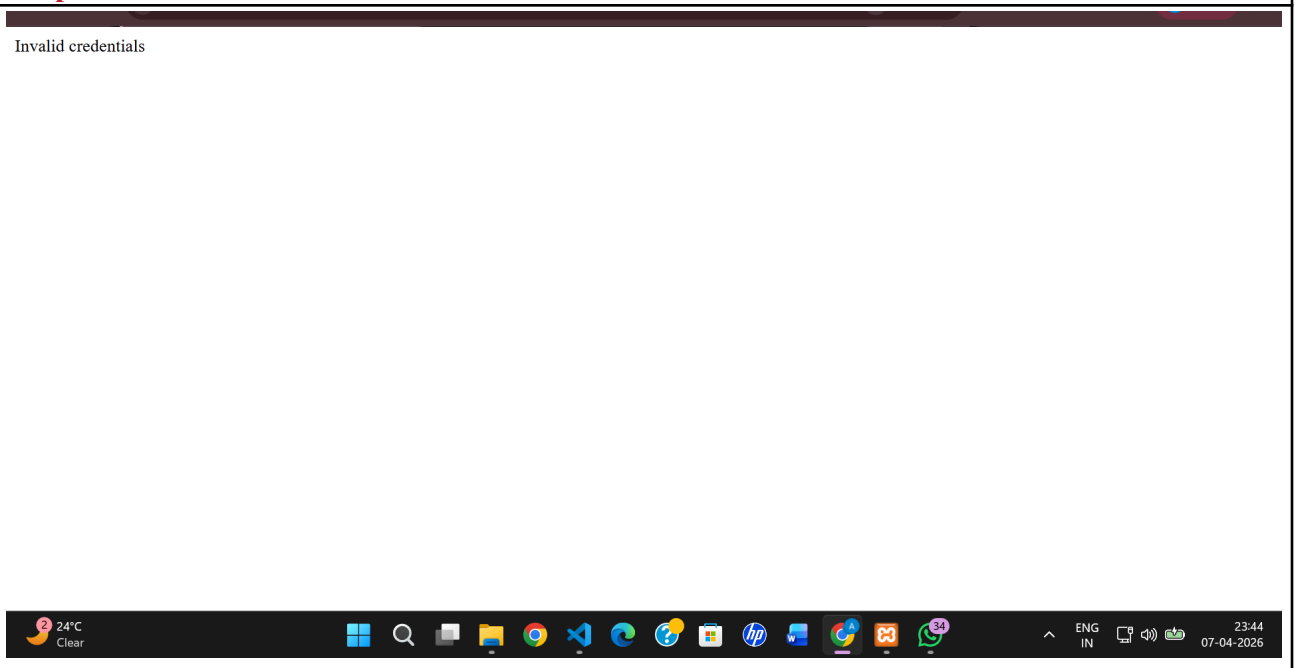
Note: Mini Project should be discussed regularly with lab faculty.

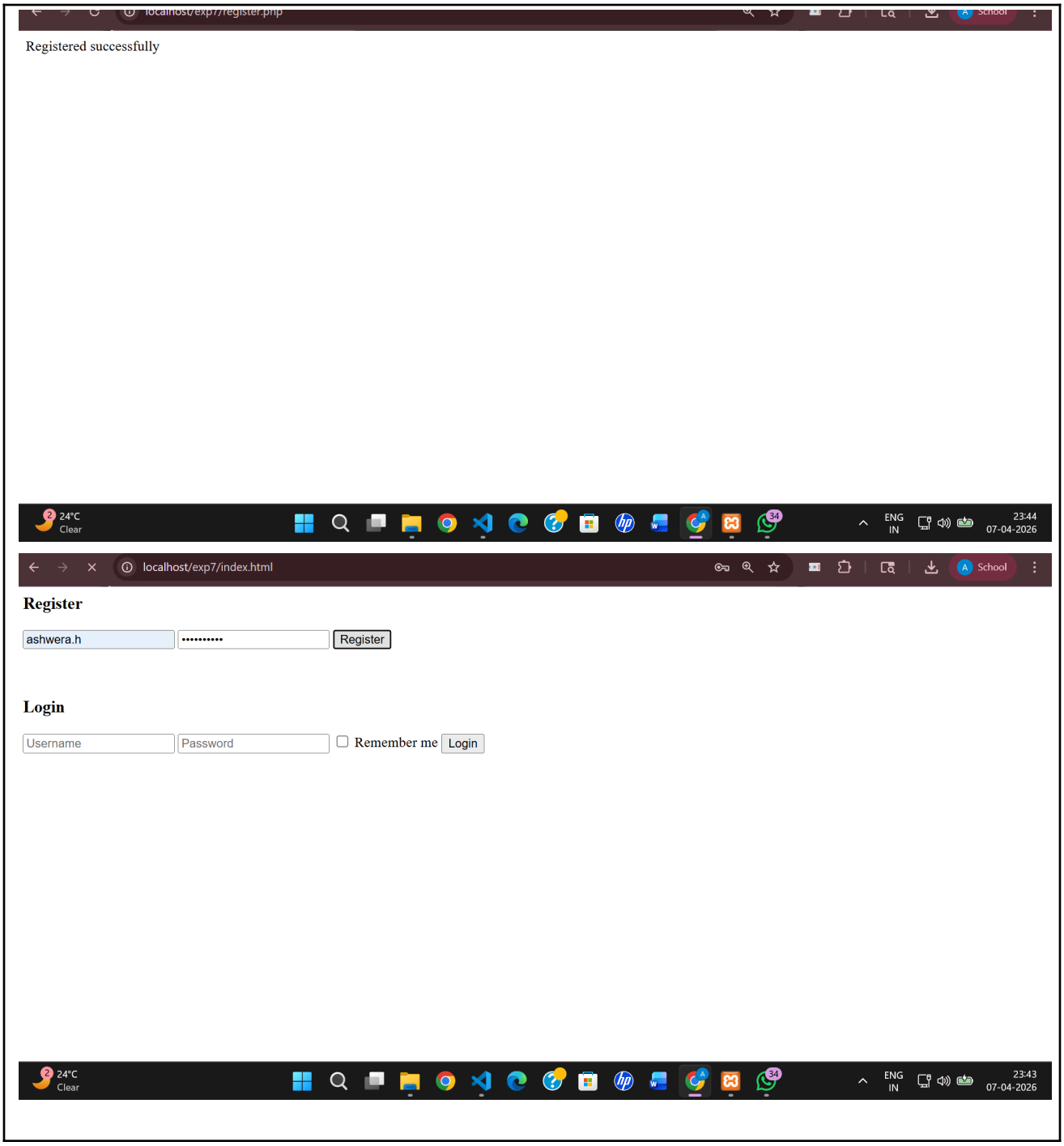
Note: Ensure all mini project code / work is regularly pushed to GitHub

Code:

Github: https://github.com/ashwera/webProgrammingLab/

Output / Screenshot:





Post Lab Subjective/Objective type Questions:

1. Design a user registration form that submits data via AJAX. Upon successful submission, display a confirmation message dynamically on the same page without reloading. Explain how you would handle error messages and form validation using jQuery.

```
<!doctype html>
```

```html
<html>
  <head>
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  </head>

  <body>
    <h3>Register</h3>

    <form id="regForm">
      <input type="text" name="username" placeholder="Username" /><br />
      <input type="password" name="password" placeholder="Password" /><br />
      <button type="submit">Register</button>
    </form>

    <p id="msg"></p>

    <script>
      $("#regForm").submit(function (e) {
        e.preventDefault();

        $.ajax({
          url: "register.php",
          type: "POST",
          data: $(this).serialize(),
          success: function (response) {
            $("#msg").text(response);
          },
          error: function () {
            $("#msg").text("Error occurred");
          },
        });
      });
    </script>
  </body>
</html>
```

```php
<?php
include 'db.php';

$username = $_POST['username'];
$password = password_hash($_POST['password'], PASSWORD_DEFAULT);

if($username=="" || $_POST['password']==""){
    echo "All fields required";
    exit();
}

$sql = "INSERT INTO users(username,password)
VALUES('$username','$password')";

if(mysqli_query($conn,$sql)){
    echo "Registered successfully";
}else{
    echo "Error";
}
?>
```

Error messages and validation can be handled using jQuery by checking input fields before sending AJAX requests. Conditions like empty fields or invalid formats are validated, and appropriate messages are displayed dynamically using selectors, preventing submission until valid input is provided.

2. Create a live search feature for a product catalog webpage. When users type in a search query, AJAX should fetch filtered product results from the server without refreshing the page. Describe how you would set up the server endpoint and handle the search input using jQuery.

```html
<!DOCTYPE html>
<html>
<head>
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>

<body>
```

```html
<input type="text" id="search" placeholder="Search product">

<div id="result"></div>

<script>
$("#search").keyup(function(){
    let query = $(this).val();

    $.ajax({
        url: "search.php",
        method: "POST",
        data: {search: query},
        success: function(data){
            $("#result").html(data);
        }
    });
});
</script>

</body>
</html>
```

```php
<?php
include 'db.php';

$search = $_POST['search'];

$sql = "SELECT * FROM products
WHERE pname LIKE '%$search%'";

$result = mysqli_query($conn,$sql);

while($row = mysqli_fetch_assoc($result)){
    echo $row['pname']." - ".$row['price']."<br>";
}
```

```
?>
```

The server endpoint is created as a PHP file that receives the search query via POST and filters products using an SQL LIKE query. On the client side, jQuery captures user input using the keyup event and sends AJAX requests to the server, updating results dynamically without page reload

3. Implement a "Load More" button for a blog homepage that fetches additional posts when clicked. Use AJAX to retrieve new content from the server and append it to the existing list without reloading the page. Discuss the necessary jQuery methods and how you would manage the loading state.

```
<!DOCTYPE html>
<html>
<head>
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>

<body>

<div id="posts"></div>

<button id="loadMore">Load More</button>

<script>
let limit = 3;
let offset = 0;

function loadPosts(){
    $.ajax({
        url: "load.php",
        method: "POST",
        data: {limit: limit, offset: offset},
        success: function(data){
            $("#posts").append(data);
            offset += limit;
        }
    });
```

```
}

loadPosts();

$("#loadMore").click(function(){
    loadPosts();
});
</script>

</body>
</html>
```

```
<?php
include 'db.php';

$limit = $_POST['limit'];
$offset = $_POST['offset'];

$sql = "SELECT * FROM posts LIMIT $limit OFFSET $offset";
$result = mysqli_query($conn,$sql);

while($row = mysqli_fetch_assoc($result)){
    echo "<p>".$row['title']."</p>";
}
?>
```

jQuery methods like click() trigger the Load More action, while $.ajax() sends requests to the server. The .append() method adds new content dynamically. Loading state is managed by disabling the button, showing a "Loading…" message, and re-enabling it after completion.

4. Develop a dashboard that fetches and displays user statistics using AJAX. When a user selects a date range, the dashboard should dynamically update the graphs without refreshing. Outline how you would implement this feature using jQuery to handle the AJAX requests and process the returned data.

```
<!DOCTYPE html>
<html>
<head>
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

```
</head>

<body>

<h3>Dashboard</h3>

From: <input type="date" id="from">
To: <input type="date" id="to">
<button id="load">Load Data</button>

<div id="stats"></div>

<script>
$("#load").click(function(){
    let from = $("#from").val();
    let to = $("#to").val();

    $.ajax({
        url: "stats.php",
        method: "POST",
        data: {from: from, to: to},
        success: function(data){
            $("#stats").html(data);
        }
    });
});
</script>

</body>
</html>
```

```
<?php
include 'db.php';

$from = $_POST['from'];
$to = $_POST['to'];
```

```php
$sql = "SELECT COUNT(*) as total
FROM users
WHERE created_at BETWEEN '$from' AND '$to'";

$result = mysqli_query($conn,$sql);
$row = mysqli_fetch_assoc($result);

echo "Total Users: ".$row['total'];
?>
```

The date inputs are captured using jQuery events, and an AJAX request is sent to the server with the selected range. The server returns filtered data, which jQuery processes and updates the dashboard dynamically using DOM manipulation without refreshing the page.

5. Discuss potential security vulnerabilities associated with the use of sessions and cookies in web applications. Specifically, explain what session hijacking is and how attackers might exploit sessions or cookies to gain unauthorized access. What countermeasures can be implemented in your web application to mitigate these risks?

   Sessions and cookies introduce vulnerabilities like session hijacking, fixation, and cross-site scripting attacks. Session hijacking occurs when an attacker steals a valid session ID, often via unsecured networks or XSS, and uses it to impersonate a user. Attackers may exploit cookies if they are not properly secured, accessing sensitive authentication data.

   To mitigate these risks, secure practices should be implemented such as using HTTPS to encrypt data, setting cookies with HttpOnly and Secure flags, regenerating session IDs after login, and implementing session timeouts. Input validation and protection against XSS and CSRF attacks are also essential to ensure application security.

6. Analyze the impact of cookie expiration settings on user experience in a web application. How would you determine the appropriate expiration time for cookies used in the "remember me" functionality? Additionally, discuss the implications of using persistent vs. session cookies for user authentication and how each affects user privacy and application security.

   Cookie expiration directly affects both user experience and security. Longer expiration improves convenience by keeping users logged in, but increases risk if cookies are stolen. Shorter expiration enhances security but may inconvenience users with frequent logins.

The appropriate expiration time for “remember me” depends on application sensitivity. For low-risk apps, longer durations may be acceptable, while sensitive systems should use shorter durations and additional verification.

Persistent cookies remain after the browser is closed, improving usability but posing higher security and privacy risks. Session cookies expire when the browser closes, offering better security but reduced convenience.

7. (Optional) Analyze the impact of cookie expiration settings on user experience in a web application. How would you determine the appropriate expiration time for cookies used in the "remember me" functionality? Additionally, discuss the implications of using persistent vs. session cookies for user authentication and how each affects user privacy and application security.

   Cookie expiration settings directly influence usability and security in web applications. Longer expiration improves user convenience by reducing repeated logins, while shorter expiration enhances security by limiting the time window for misuse if cookies are compromised.

   The appropriate expiration time for “remember me” functionality should be chosen based on application sensitivity and user expectations. Less sensitive systems may allow longer durations, whereas secure systems should use shorter durations with additional verification mechanisms.

   Persistent cookies remain stored even after the browser is closed, offering better user experience but increasing risks related to privacy and unauthorized access. Session cookies expire when the session ends, providing stronger security but requiring users to log in again, thus reducing convenience.

8. (Optional) Analyze the impact of cookie expiration settings on user experience in a web application. How would you determine the appropriate expiration time for cookies used in the "remember me" functionality? Additionally, discuss the implications of using persistent vs. session cookies for user authentication and how each affects user privacy and application security.

   Cookie expiration settings affect both usability and security in web applications. Longer expiration improves user experience by reducing frequent logins, while shorter expiration enhances security by limiting the risk if cookies are compromised or stolen.

The appropriate expiration time for “remember me” functionality depends on application sensitivity and user needs. Low-risk systems may use longer durations, while high-security applications should use shorter durations along with additional authentication measures.

Persistent cookies remain stored after the browser closes, improving convenience but increasing privacy and security risks. Session cookies expire when the browser closes, offering better security but requiring users to log in again, reducing overall user convenience

Conclusion and Discussion:

The experiment successfully demonstrated session management and cookie handling for user authentication in a web application. It provided understanding of maintaining user state, improving user experience, and implementing basic security practices using PHP, MySQL, and AJAX techniques.
Riya: The implementation works well and demonstrates sessions clearly, but UI can be improved.
Kushal: Good use of cookies and authentication, though more validation can be added.
Navya: The application is functional and easy to understand, but security features can be enhanced further.